

Component A

1. Interview scripts and notes

Current Return Process

- Teams come to a table on the third floor to return items
- Staff go through a checklist one by one with each team, confirming items and marking them off on the return list
- After check-in, Kevin and the speaker divide items into categories: makerspace, IT, or items that don't belong in either
- Asset tags (8-digit barcode numbers provided by University of Washington) are added to items
- Items are then entered into BlueTally, the software used to track checkouts
- The process is time-consuming due to the volume of items and the need to process each item individually

Key Challenges

- The return process is slow and requires going through items one by one with each team
- Teams must check in twice: first with physical items against a list, then again in BlueTally
- Asset tags must be added manually to items before they can be entered into BlueTally
- Items purchased by teams don't have barcodes when purchased, but need them added during return
- Amazon product descriptions are overly long and need to be cleaned up into proper product names (e.g., "Hollyland Lark M2 wireless microphone" instead of full Amazon title)
- Some items are too small for physical barcode labels, requiring workarounds like placing labels on boxes

BlueTally System Overview

- BlueTally tracks all checkable assets including laptops, headsets, cameras, microphones, and other equipment

- The system distinguishes between "assets" (individual items with unique barcodes) and "accessories" (interchangeable items like cables that don't need individual tracking)
- BlueTally has a CSV import function for adding assets
- Basic information needed for each asset includes: asset number (SID), product name, status, and location
- Students can access JxHub to view available assets, but checkouts should go through staff
- BlueTally has API capabilities for automation, but they require paid access
- Alan has written a script using the API to pull overdue items and send reminder emails

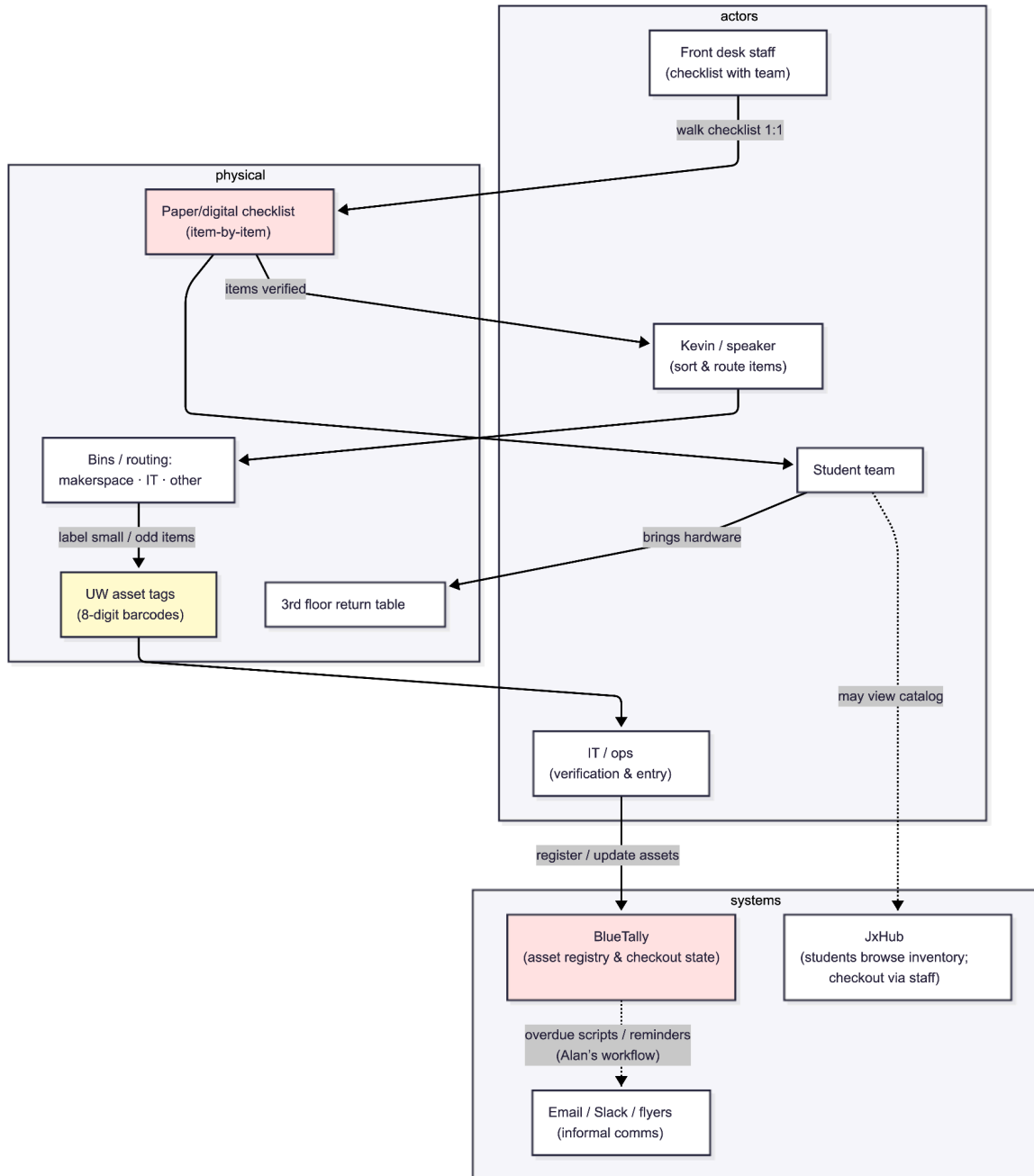
Potential Solutions Discussed

- Use CSV upload to bulk add asset tags and items to BlueTally
- Get the barcode scanner working more effectively with the system
- Modify the purchase workflow to add barcodes immediately when items arrive, rather than waiting until return time
- Consolidate the dual check-in process into a single BlueTally-based check-in
- Use UPC barcode databases to automatically pull product information (title, description) when scanning items
- Streamline product name entry by parsing Amazon descriptions into cleaner product names

Return Verification and Edge Cases

- Items should be returned directly to IT staff for verification
- Items left unattended (e.g., on desks) sometimes create gaps in tracking
- Multi-part items like cameras require checking inside cases to ensure all components are returned
- Items that are consumed during prototyping don't need to be returned
- Reusable items are categorized and kept for future use
- Projects are kept together when possible to show to prospective students

2. System map sketch — boxes and arrows showing the **interviewee's** tools and systems, data flows, and pain points, with system touchpoints marked (hand-drawn photo or digital diagram)



Component B

1. **Tech stack justification** — 1 sentence explaining why you chose Streamlit or Next.js for this problem (from your [Spec Checkpoint](#))

I chose Streamlit + Supabase because this is a small staff-facing internal tool with rapid form entry, CSV export, and a second events page—Streamlit let me ship Python + one repo quickly without building auth and a full React UI, while Supabase still gives shared, persistent data for both return lines and the events list.

2. **Supabase schema report** — table/schema details for your data model (table names + key fields; screenshot or SQL is fine)

`return_lines` (Component B — return intake / BlueTally-style export)

- a. `id` (uuid, PK)
- b. `amazon_title` (text, required) — raw vendor/Amazon title
- c. `cleaned_name` (text, required) — short name for import
- d. `barcode` (text, optional) — optional UPC for Open Food Facts lookup
- e. `asset_sid` (text, optional) — asset / SID if known
- f. `status` (text, default `pending_import`)
- g. `location` (text, default `GIX`)
- h. `created_at` (timestampz)

3. **Responsive design summary** — completed checklist table from the [Responsive Design Check](#) (or a short bullet list), including **one** issue you found at phone width and **what you changed** to fix it

Equipment return intake (Component B)

Stores rows in Supabase for CSV export; optional barcode lookup uses free Open Food Facts.

Amazon / vendor title (long OK)

Optional barcode digits (Open Food Facts lookup)

Optional asset SID / tag

Cleaned product name (BlueTally-friendly)

Save line to Supabase

Saved rows → CSV (BlueTally-style columns)

	cleaned_name	asset_sid	status
0	Apple Macbook Pro 2018 intel 13' Grey	None	pending_import

Download CSV

Issue: On a narrow viewport, body text and captions were too light, and action buttons were too small for comfortable tapping.

Added shared styles in `lab5_yilu/lib/ui.py` (injected after `set_page_config`) to increase contrast and font size, enlarge button hit targets, and set `lab5_yilu/Home.py` primary actions (Save, Download CSV) to `type="primary"` with `use_container_width=True`.

Element	Works at Phone Width?	What Breaks?
Page title -- is it fully visible?	Yes	Title was readable; no change required for truncation.
Navigation -- can you access all pages/sections?	Yes	Streamlit sidebar can be toggled on narrow screens
Forms -- can you fill out and submit forms?	Yes	Before: Primary actions felt too small to tap. After: "Save" uses primary + full width; fields remain usable.
Tables -- can you read table content? (horizontal scrolling?)	Mostly Yes	Before: Dataframe text felt tiny. After: CSS targets <code>stDataFrame</code> for larger, darker text; very wide tables may still scroll horizontally (expected on mobile).
Charts -- are they readable? (labels visible, hover works?)	N/A	This app does not use charts on the main flows.
Buttons -- are they large enough to tap with a finger?	Yes	Before: Buttons were small. After: min-height ~3rem, larger padding, primary style, <code>use_container_width</code> on save/download.

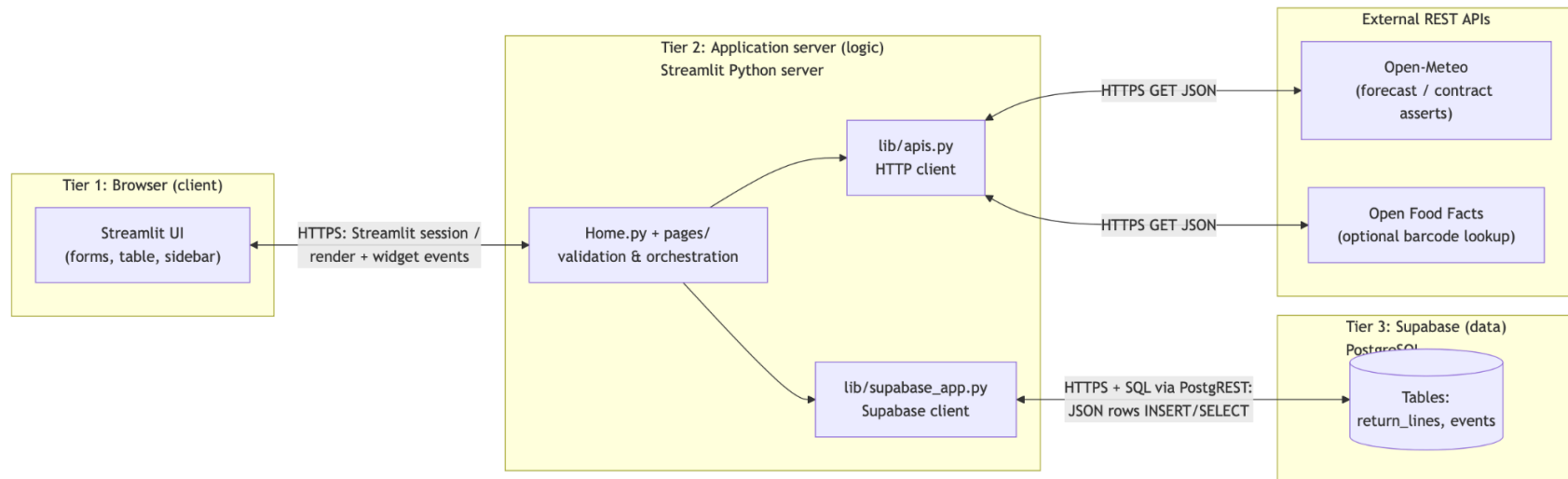
Text -- is the font size readable without zooming?	Yes	Before: Default gray body/caption text had low contrast and small size. After: lib/ui.py sets darker text (#141414 / #2d2d2d) and ~1.05rem base sizing.
--	-----	---

4. **Deployment URL** — live link to your deployed app (Vercel or Streamlit Cloud) and a note confirming you checked that no secrets are exposed

<https://510lab5-xwdx8ajp8kmgrpknw8qzl.streamlit.app>

Component C

1. **Architecture diagram** from the [C.2](#) exercise for your **Component B** app (hand-drawn photo or digital). This is the **3-tier** view (browser, app server, Supabase + external API).
 - Tier 1 — Browser (client): User interacts with the Streamlit UI (Home.py, multipage pages/), forms, tables, sidebar weather widget.
 - Tier 2 — Application server (logic): Streamlit server executes Python: lib/supabase_app.py (Supabase client), lib/apis.py (HTTP calls to external REST APIs), lib/text_clean.py, validation/error handling in UI callbacks.
 - Tier 3 — Supabase (data): PostgreSQL tables return_lines (and events used elsewhere); persists rows across sessions.
 - External APIs (sidecar): Open-Meteo (weather), Open Food Facts (optional barcode); called only from Tier 2 (server-side Python), not from the browser directly.



2. Design Decision Log entry from C.3

Field	Your entry
Decision	Wrap Supabase inserts/selects and external API calls in try/except (and Streamlit st.error / st.warning) so failures show an in-app message instead of crashing the session.
Alternatives considered	Fail silently; retry forever; redirect user to a blank page.
Why you chose this	Lab explicitly requires error handling on every API/DB path; Streamlit reruns make uncaught exceptions disruptive during demos.
Trade-off	Users see generic messages unless you invest in tailored copy per error code.

When would you choose differently?	For production, add structured logging, retry with backoff for 429/5xx, and RLS policies instead of turning security off.
---	---

C.2 vs. Component E: The **C.2** diagram is only for the **integrated API + Supabase app** from Component B. **Component E** requires a **separate** system architecture map (Part 1) with boundary labels and errors — do not substitute one for the other.

Component D

1. **Validation results** from D.1 exercise (completed recording template with 3 scenarios: valid, invalid, and missing-auth or third invalid variant)

#	Test Case	Input Description	Expected Outcome	Actual Outcome	Status Code	Pass/Fail
1	Valid input	latitude=47.6588, longitude=-122.3135, current_weather=true	200 OK, JSON containing current_weather with temperature	200, JSON includes current_weather.temperature	200	Pass
2	Invalid input	latitude=999 (out of valid Earth range for their API validation), longitude=-122.3135, current_weather=true	Request rejected or error payload — not a normal forecast body	typically 400 Bad Request or API returns an error field / empty usable forecast	400	Pass

3	Third invalid variant	Wrong URL path, e.g. GET https://api.open-meteo.com /v1/forecastZZ with same params	404	404 + HTML or short error body	404	Pass
---	-----------------------	---	-----	--------------------------------	-----	------

2. **External API assert statements** — assert checks for your Component B external API response (see [API response asserts](#)).

Location: `lab5_yilu/lib/apis.py`, function `fetch_open_meteo_current`.

After a successful HTTP response, the code checks the JSON contract:

```
assert "current_weather" in data
assert isinstance(data["current_weather"], dict)
assert "temperature" in data["current_weather"]
assert isinstance(data["current_weather"]["temperature"], (int, float))
```

3. **Error-handling note** — brief note (README or docs) on whether each D.1 scenario was handled gracefully in your app or revealed a gap

The app catches failures from external APIs and Supabase instead of crashing the Streamlit session. Open-Meteo in the sidebar uses try/except and shows st.error with the exception text. Optional Open Food Facts barcode lookup uses try/except and shows st.warning. Supabase insert and reload paths use try/except with st.error.

For D.1: the valid Open-Meteo case matches normal use (fixed Seattle-area coordinates). The invalid-parameter and third invalid scenarios were validated with curl/off-app requests; the UI does not expose controls to send bad coordinates, so users cannot reproduce those rows from the page alone—however any failure inside the sidebar try block would still surface as st.error rather than an uncaught exception.

Gap: there is no in-app control to replay every D.1 scenario; contract checks are partly validated outside the Streamlit UI.

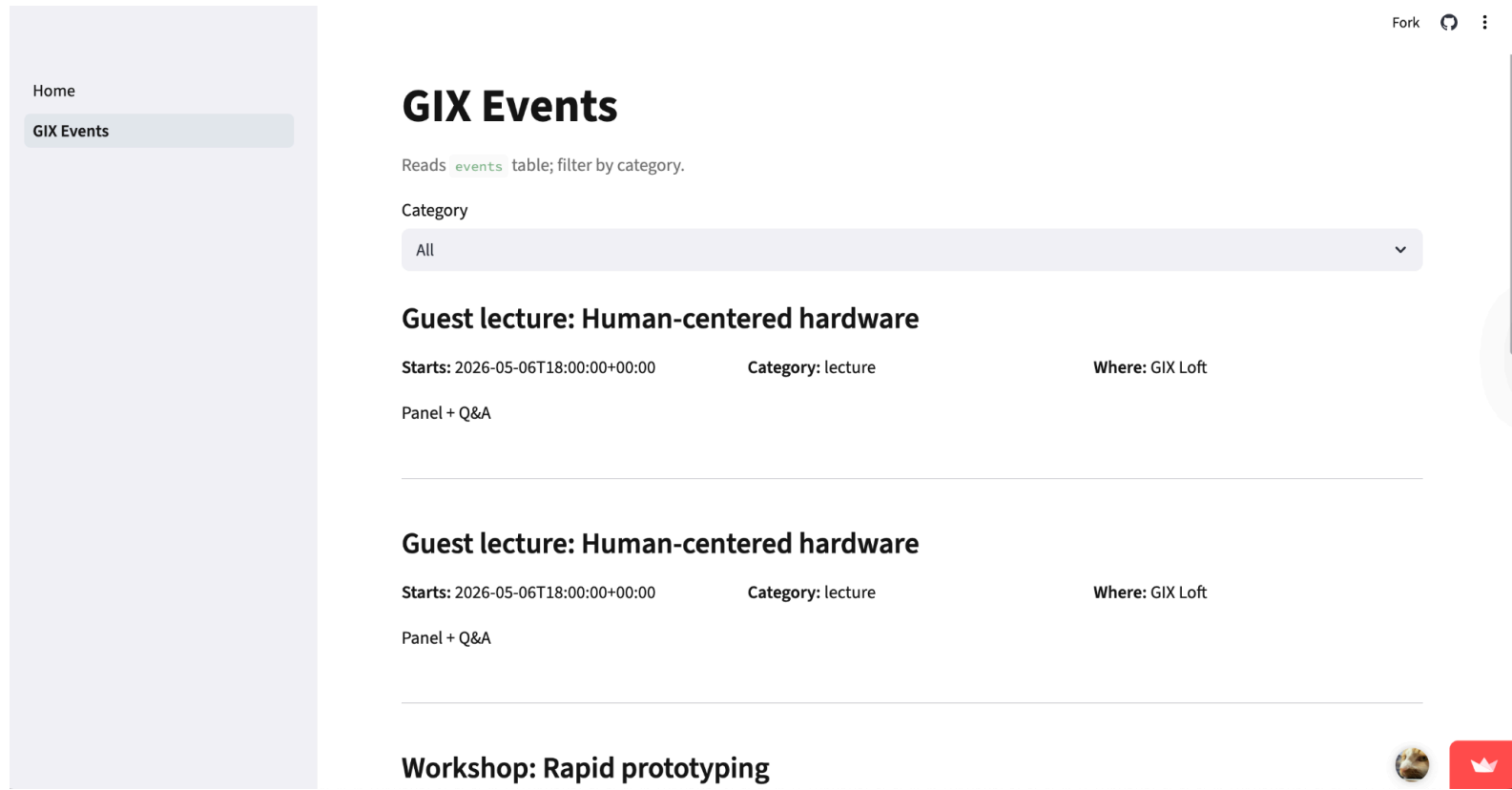
Component E

Submit as part of the **same repo**.

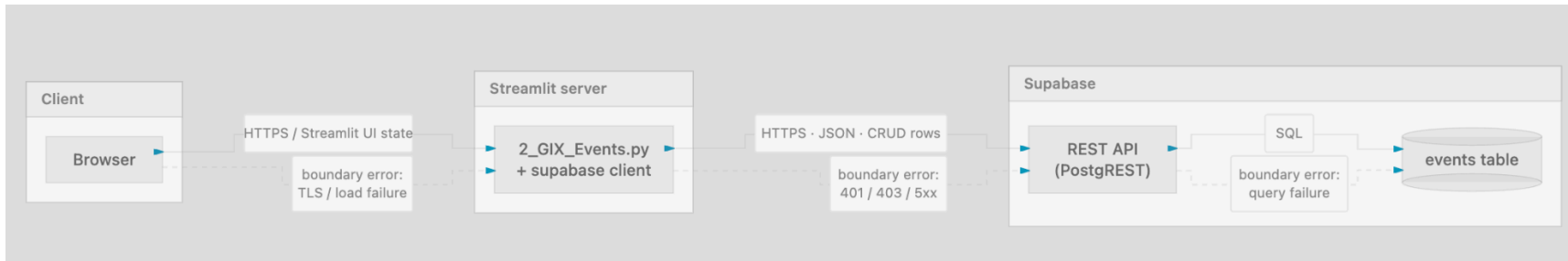
1. **Working events UI** — code that reads from your **events** Supabase table and shows a **category filter** (screenshot in README is fine)

****Code:**** `lab5_yilu/pages/2_GIX_Events.py` — reads Supabase table

****events****, filters by ****category**** via `st.selectbox`.



2. **System architecture map** from **Component E** → **Part 1: Architecture & Design** (image/PDF in repo or attached on Canvas) with **data format** and **one potential error** labeled at each boundary



3. **Testing write-up** (README section or docs/component-e.md): **3** error/failure scenario tests (what you did, expected, actual) + **2** assert statements for the **events/Supabase** pipeline (snippet or file path)

Scenario A — Invalid or missing Supabase credentials

Item	Description
What I did	Temporarily replaced <code>SUPABASE_KEY</code> in <code>lab5_yilu/.env</code> with an incorrect value (or used a wrong key in Streamlit Secrets when testing deployed app). Restarted the app and opened the GIX Events page. Restored the correct key after the test.
Expected	The app cannot load events; the user sees an error message. No secret value is printed in the UI.
Actual	<i>Could not load events (handled): ...</i>

Scenario B — Empty result set / filter with no matching rows

Item	Description
What I did	In the GIX Events page, selected a Category value that has no events in Supabase

Expected	No crash; an informational message indicates there are no events for the current filter.
Actual	<i>No events for this filter. Insert sample rows in Supabase Table Editor.</i>

Scenario C — Database / network failure when querying

Item	Description
What I did	Briefly used an invalid table name in code locally without committing to reproduce a query failure.
Expected	Exception is caught; user sees <code>st.error</code> with a handled message, not a blank crash.
Actual	<i>"Could not load events (handled): ..." with Postgres/REST error text</i>

Assert 1 — Line 35: `assert isinstance(rows, list), "events query must return a list"`

The value returned from Supabase for this query must be a Python list (empty list if there are no rows). If the client ever returned a non-list payload, the app fails fast before building categories.

Assert 2 — Line 36: `assert all(isinstance(r, dict) for r in rows), "each event row must be a dict"`

Every row in that list must be a dictionary (JSON object shape), matching what supabase-py normally returns for each record. If any element were not a dict, rendering would be unsafe or inconsistent.

4. **Security** — confirm where secrets live; no keys in source (align with the [Security Checklist](#) and E Part 3)
 - Secrets location: Local development uses `lab5_yilu/.env` (gitignored) with `SUPABASE_URL` and `SUPABASE_KEY`.
 - Deployment: Streamlit Community Cloud → App Secrets (same variable names) — not committed to GitHub.

- Source check: No Supabase keys or tokens are hardcoded in .py files; only os.environ / st.secrets via lib/supabase_app.py.

AI Usage Log

1. **What you prompted:** Deploy my Streamlit app from a private GitHub org repo to Streamlit Cloud; it says I don't have access / 404.
 - **What it produced:** Steps to grant Streamlit GitHub App access to the private repo, use blob/.../Home.py URL or interactive picker, set Secrets for Supabase, optionally mirror repo under personal GitHub.
 - **AI assumption:** That your GitHub account already has org permission to install/configure the app; that the failure was access not a typo in the app URL.
 - **Failure mode:** You keep failing if an org admin must approve third-party access; you might paste the wrong branch in the blob URL.
 - **What you would change:** Add: org name, exact error text, and whether Streamlit is already in Installed GitHub Apps; ask: "Do I need org approval?" first.
2. **What you prompted:** The app shows 'Supabase not configured' even after I copied URL and anon key. The .env file exists.
 - **What it produced:** Check .env path next to Home.py; ensure SUPABASE_URL / SUPABASE_KEY names match code; restart Streamlit after edits; load .env from a fixed project root so cwd doesn't matter; on Streamlit Cloud, put the same keys in Secrets because .env isn't on GitHub.
 - **AI assumption:** That the keys were valid and the issue was loading/path, not an expired or truncated key.
 - **Failure mode:** Path/load_dotenv while the real issue is a bad paste or RLS/403 mistaken for "not configured."
 - **What you would change:** Add: "Variables print empty in a one-line Python check" or "Supabase Table Editor insert works"; separate local vs Cloud in the prompt.
3. **What you prompted:** On narrow screen, buttons feel too small, text too light. Safari — how do I open responsive mode without F12?

- **What it produced:** Safari: enable Develop menu → Enter Responsive Design Mode; shortcuts ⌘⇧I; suggested shared CSS (inject_accessible_theme) for darker text, larger tap targets, type="primary" + use_container_width on save/download.
- **AI assumption:** That Streamlit default styling was the main issue rather than browser zoom or OS accessibility settings.
- **Failure mode:** Global CSS might not hit every Streamlit inner component after an upgrade; some widgets stay small until Streamlit changes DOM.
- **What you would change:** Specify Streamlit version, attach a screenshot, ask for minimal CSS scoped to stButton / stDataFrame only if something still breaks.

Reflection

I used Streamlit end-to-end instead of Next.js, so the biggest difference I felt was speed of layout and forms in Python versus splitting UI, routes, and env in a React stack, easier to ship a two-page demo, harder to match polished SPA patterns without extra components. Reading AI-generated Python still feels linear (one rerun model), whereas AI-generated TypeScript in Next leaves more files and boundaries to trace when something breaks. Drawing the system map showed double check-in and CSV/BlueTally steps more clearly than Q&A alone, the pain was coordination between desk work and database entry, not only “returns are slow.” For stack choice, Streamlit fits internal staff tools and rapid CSV/database workflows (our return intake prototype), while Next.js + Supabase fits multi-user, auth-heavy, customer-facing apps (e.g., a public events portal with accounts).